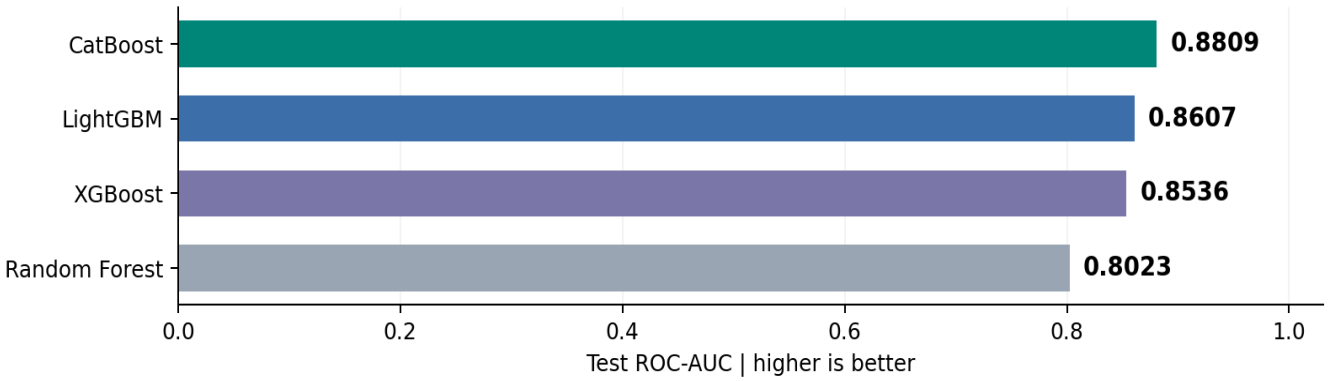
Amazon Employee Access

Four tuned tree models | Test results and interpretation

CatBoost provides the strongest prediction quality and the fastest measured batch inference among the four tuned models.

Best test ROC-AUC	Lowest test log loss	Fastest inference
CatBoost: 0.8809	**CatBoost: 0.1426**	**CatBoost: 19.90 ms**



Model	Test ROC-AUC Higher is better	Test log loss Lower is better	Denial AP Higher is better
CatBoost	0.8809	0.1426	0.5212
LightGBM	0.8607	0.1527	0.4789
XGBoost	0.8536	0.1584	0.4443
Random Forest	0.8023	0.1836	0.3234

Main findings

Quality: CatBoost exceeds LightGBM by 0.0203 ROC-AUC and reduces log loss by **6.6%**. Its denial AP is 0.5212 versus 0.4789 for LightGBM.

Speed: CatBoost predicts 1,000 rows in **19.90 ms**, including input preparation. LightGBM takes 48.88 ms. CatBoost is about **2.46 times faster** for this batch workload.

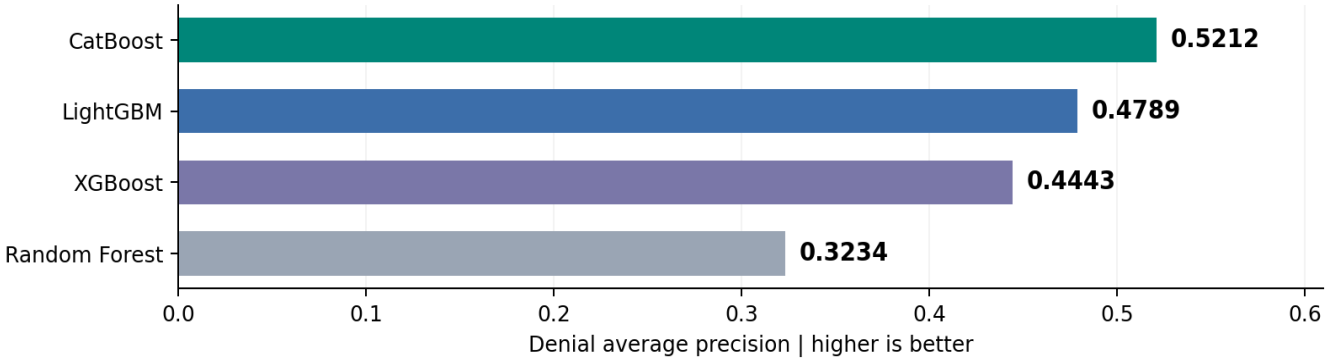
Evidence: CatBoost was selected by validation log loss before test evaluation. Paired test bootstrap intervals support lower log loss than each alternative.

Design: 32,769 labelled rows; 22,938 training, 4,915 validation and 4,916 test. Nine categorical predictors. Random Forest uses one-hot encoding; the other models use native handling. Approximately 10 minutes of Optuna tuning per model; four CPU threads.

Prediction quality

Probability quality and detection of denied requests

Only **5.8%** of test requests are denied: 285 denials versus 4,631 approvals. An always-approve rule already achieves about 94.2% accuracy. Accuracy and approval F1 therefore provide an incomplete view of performance.



Denial AP summarizes precision-recall performance while treating denied requests as the positive class. CatBoost ranks denied requests most effectively. The prevalence baseline is approximately 0.058. AP is not identical to trapezoidal PR-AUC.

Model	Denial precision	Denial recall	Denial F1	Accuracy
CatBoost	0.667	0.323	0.435	95.14%
LightGBM	0.723	0.256	0.378	95.12%
XGBoost	0.677	0.235	0.349	94.91%
Random Forest	0.520	0.137	0.217	94.26%

At the saved threshold, CatBoost identifies 92 of 285 denials (32.3% recall), with 66.7% denial precision. LightGBM has higher denial precision, 72.3%, but detects fewer denials, 25.6%.

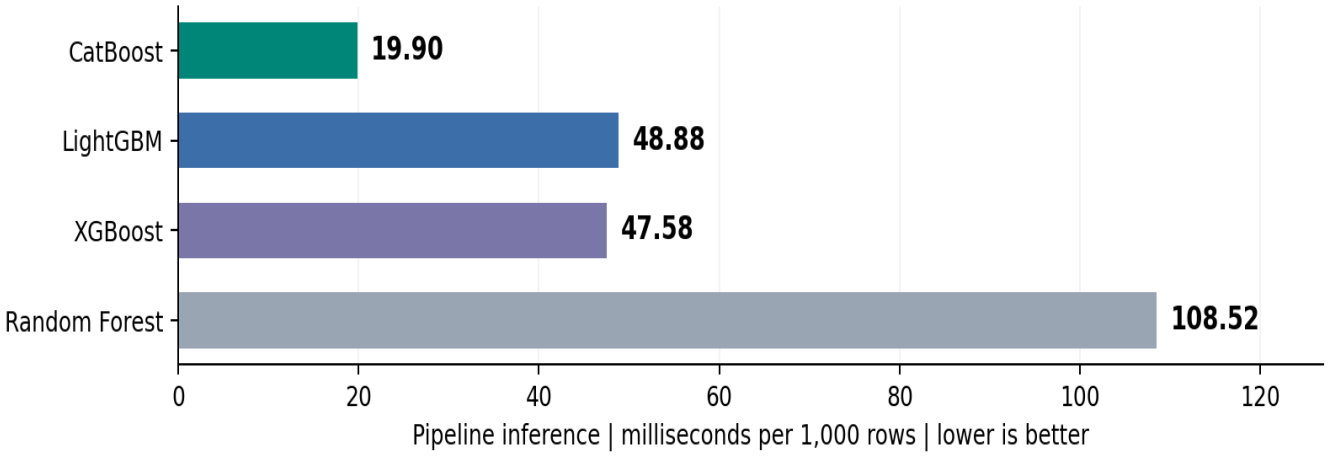
Thresholds were chosen on validation data to maximize **approval F1**. They were not optimized for denial detection. A deployment decision would require a validation-based threshold reflecting the cost of false approvals and false denials.

How to read the main metrics

Metric	Plain interpretation
ROC-AUC	Measures ranking across thresholds; 0.5 is chance ranking and 1.0 is perfect ranking. It is not classification accuracy.
Log loss	Measures probability quality and penalizes confident mistakes. Lower is better; this is the primary selection metric.
Brier score	Mean squared probability error. CatBoost has the lowest value: 0.0370, versus LightGBM at 0.0396.
F1	Balances precision and recall at the chosen threshold. Values depend on which class is positive and which threshold is used.

Inference performance

Prediction time including input preparation



Model	Pipeline inference ms / 1,000 rows	Speed relative to CatBoost
CatBoost	19.90	Fastest
LightGBM	48.88	2.46 times as long
XGBoost	47.58	2.39 times as long
Random Forest	108.52	5.45 times as long

Measurement conditions

Inference uses the same 1,000-row batch for every model, with two warm-up calls, seven timed repetitions and four CPU threads. Values are median batch times from the final consolidated notebook table.

The measured pipeline includes input copying, one-hot encoding or category conversion, and probability prediction. Disk reads and model loading are excluded.

Interpretation

CatBoost is fastest at 19.90 ms per 1,000 rows. XGBoost follows at 47.58 ms and LightGBM at 48.88 ms. Random Forest takes 108.52 ms, including its one-hot preprocessing.

CatBoost combines the strongest test probability quality with the fastest measured inference. XGBoost and LightGBM have similar observed batch times; repeated-run variability should be considered before treating their small timing gap as meaningful.

These measurements describe warmed-up batch prediction on this computer. They are not single-request latency measurements and do not establish speed rankings for every hardware configuration or batch size.

Evidence and interpretation

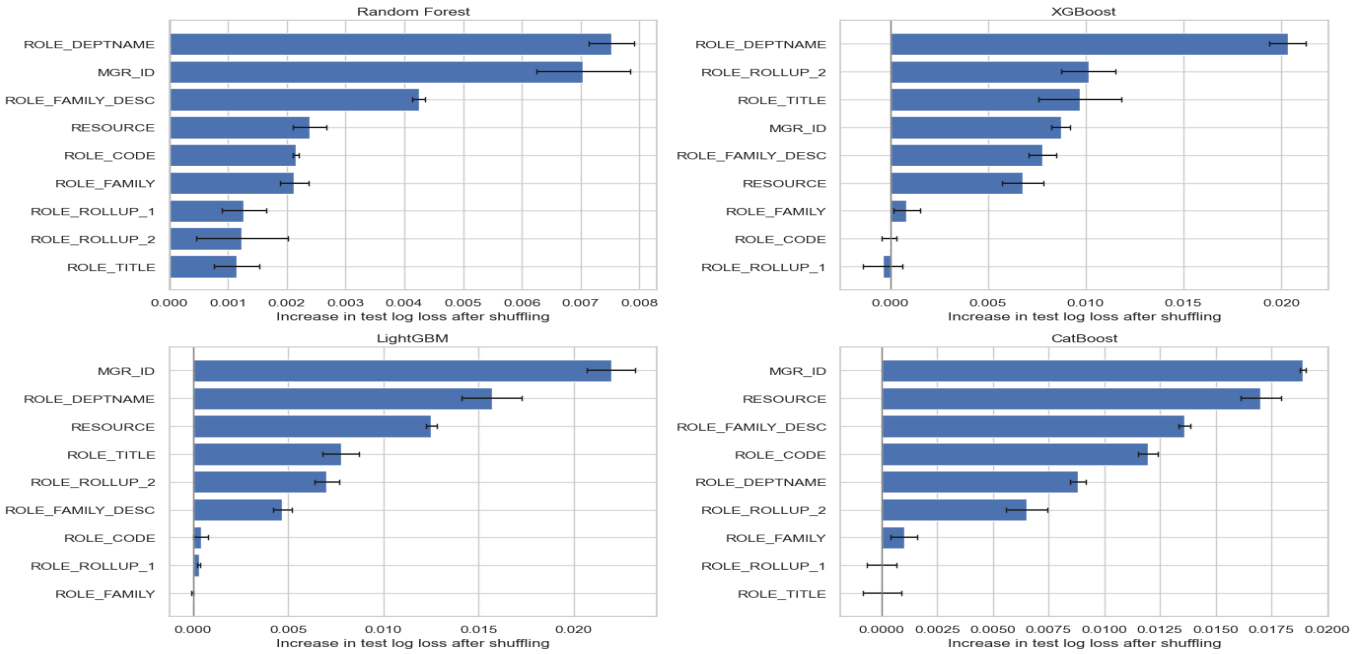
A clear advantage on this split, with a defined scope

Paired bootstrap evidence for log loss

Comparison	Loss difference	95% interval
LightGBM minus CatBoost	+0.0101	[+0.0052, +0.0158]
XGBoost minus CatBoost	+0.0158	[+0.0100, +0.0220]
Random Forest minus CatBoost	+0.0410	[+0.0329, +0.0490]

All three intervals lie above zero: CatBoost has lower log loss than each alternative. The calculation uses 1,000 paired, class-stratified bootstrap resamples of the same test rows. These intervals reflect test-sample uncertainty for fixed models, not variability from retraining; they are not adjusted for multiple comparisons.

Which features matter?



CatBoost relies most on **MGR_ID** and **RESOURCE**, followed by ROLE_FAMILY_DESC and ROLE_CODE. Shuffling these columns raises test log loss. Importance uses the same original-column permutation method across models; it indicates predictive reliance, not causation. Correlated features can share or mask importance.

Conclusion

CatBoost is the strongest overall prediction choice in this Amazon experiment. It combines the best test ranking, probability quality and denial AP with the fastest measured batch inference. The result applies to this dataset, tuning budget and measured batch workload.

This result supports CatBoost for this categorical access-control task. It does not establish that native handling alone caused the advantage: algorithms, tuning spaces and representations differ. One split, one training seed and a limited search budget do not establish a universal winner.

Source: executed outputs and plots in 03_amazon_evaluation(1).ipynb. Split, metric, timing and bootstrap settings follow the notebook. No models were retrained for this report.

Why training takes longer

The cost of learning categorical relationships

CatBoost required more fitting time in this experiment. That observation is consistent with additional categorical processing, but the timing totals alone cannot isolate each source of cost.

1. Categorical statistics

All nine predictors are categorical IDs, including manager, resource and role identifiers. CatBoost received these as categorical strings. During training it learns target-based categorical statistics rather than receiving a fixed one-hot matrix. Calculating those statistics adds work.

2. Feature combinations allowed in the search

The search included **max_ctr_complexity from 1 to 2**. This allowed Optuna to select statistics for individual categorical features or combinations of up to two. Combinations can capture relationships such as a manager-resource pairing, but generating and evaluating them can increase training cost.

The search allowed combinations; the displayed evaluation outputs do not identify whether the winning configuration selected 1 or 2. Combinations therefore remain a possible contributor, not a confirmed explanation for the entire timing gap.

3. Trees, depth and the chosen configuration

The search allowed tree depths from **4 to 8** and up to **2,000 boosting iterations**, with validation early stopping after 50 rounds without improvement. The final model was fitted using the selected tree count. Deeper trees and more iterations generally require more computation.

The final fit used four CPU threads and Bayesian bootstrap. These were deliberate experiment settings. The models were not constrained to identical tree counts or identical tree structures, so fitting time reflects both the algorithm and the selected configuration.

4. Why inference can still be fast

Training searches for useful relationships and estimates the model. Inference applies the relationships and trees that have already been learned; it does not repeat the training search. Additional learning work can therefore coexist with fast prediction.

What the result supports

The experiment demonstrates a quality-versus-training-cost trade-off, alongside fast inference. It does not prove that a particular categorical mechanism caused the quality gain or the fitting-time difference. Isolating the cause would require controlled comparisons with combinations disabled or other settings held fixed.

Technical reference: CatBoost documentation, "Speeding up the training". The documentation identifies categorical statistics, categorical combinations, iteration count and other settings as contributors to training cost. https://catboost.ai/docs/en/concepts/speed-up-training